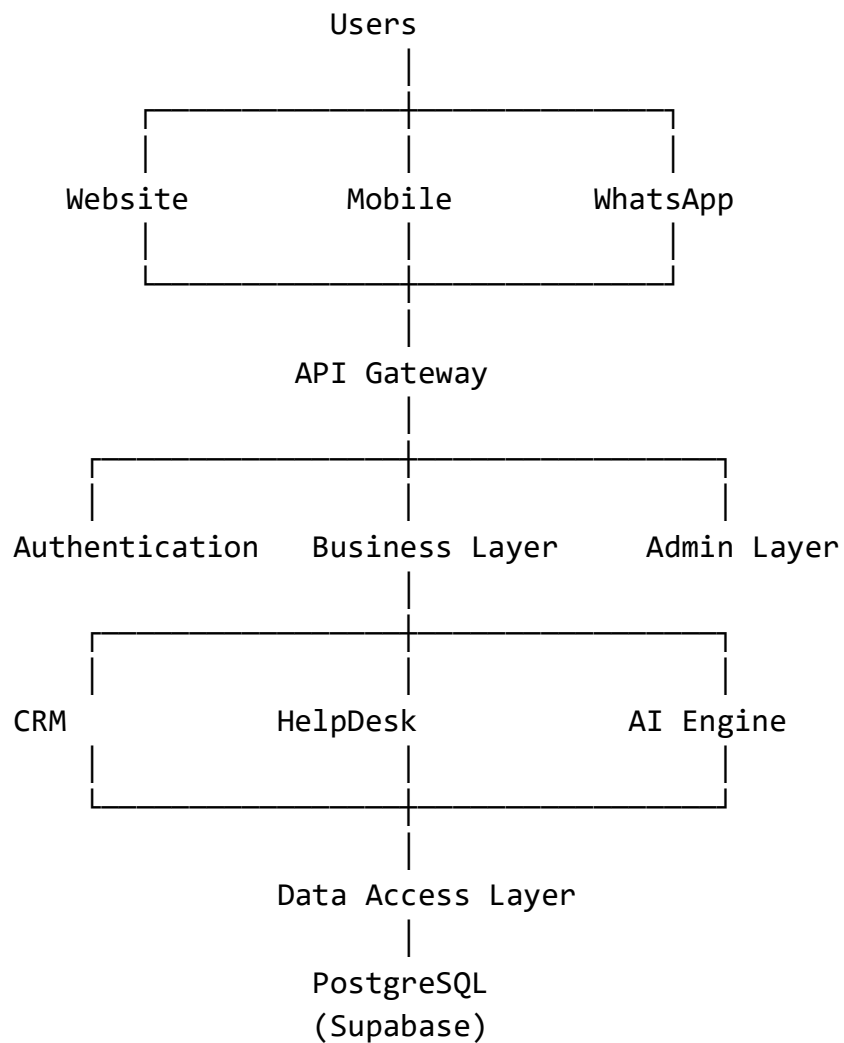


BiteFixes Enterprise Reference Architecture (BERA) v2.0

Capas del sistema



Dominio CRM

Aquí vive toda la información del cliente.

Company
|
Customers
|
Devices
|
Addresses

Cada cliente puede tener múltiples dispositivos.

Cada dispositivo tendrá su propio historial.

Ejemplo:

Juan Pérez

Notebook Dell Latitude

↓

3 reparaciones

↓

2 mantenimientos

↓

1 garantía

Esto será una gran ventaja para Bitey.

Dominio HelpDesk

Separaremos claramente:

Conversation

≠

Ticket

Una conversación puede terminar sin ticket.

Un ticket puede contener decenas de mensajes.

Conversation

↓

Diagnosis

↓

Ticket

↓

Repair

↓

Closed

Dominio IA

Aquí está el verdadero corazón de Bity.

No será un único motor.

Será un conjunto de motores especializados.

Language Engine

Intent Engine

Memory Engine

Knowledge Engine

Recommendation Engine

Response Engine

Learning Engine

Cada motor será completamente independiente.

Learning Engine

Aquí aparece una de las funciones más importantes.

Cada conversación ayudará a mejorar la IA.

Cliente pregunta

↓

Técnico responde

↓

Cliente acepta

↓

Caso cerrado

↓

Learning Engine registra

↓

Nuevo conocimiento

Así Bity podrá aprender continuamente sin modificar el código.

Dominio Knowledge

No quiero una simple tabla.

Quiero una verdadera base de conocimiento.

Knowledge Article

↓

Category

↓

Intent

↓

Language

↓

Services

↓

Tags

↓

Priority

↓

Attachments

Cada artículo podrá existir en:

- Español
- Portugués
- Inglés

Dominio Services

No será solamente un listado.

Cada servicio tendrá:

Service

↓

Category

↓

Estimated Time

↓

Base Price

↓

Warranty

↓

Requirements

↓

Recommended Products

Cuando Bitey recomiende un SSD, podrá indicar el servicio relacionado.

Dominio Devices

Este será uno de los puntos fuertes de BiteFixes.

Customer

↓

Device

↓

Brand

↓

Model

↓

Serial Number

↓

Purchase Date

↓

Warranty

↓

History

Cada reparación quedará asociada al equipo.

Dominio Analytics

No solo mediremos ventas.

Mediremos toda la plataforma.

Conversations

↓

Tickets

↓

Response Time

↓

Customer Satisfaction

↓

AI Accuracy

↓

Revenue

↓

Growth

Esto permitirá crear dashboards muy completos.

Dominio Billing (Preparado para el futuro)

Aunque inicialmente no se implemente, la arquitectura debe prever:

Plans

Subscriptions

Invoices

Payments

Coupons

Usage

Así BiteFixes podrá comercializarse como SaaS.

Eventos de dominio

Quiero que la plataforma funcione mediante eventos internos.

Ejemplos:

CustomerCreated

CustomerUpdated

ConversationStarted

MessageReceived

TicketOpened

TicketAssigned

TicketClosed

KnowledgeUpdated

ServiceCreated

Estos eventos facilitarán añadir automatizaciones sin modificar el resto del sistema.

Arquitectura de IA

No quiero que Bitey conozca Supabase directamente.

El flujo será:

Bitey

↓

Business Services

↓

Repositories

↓

Supabase

De esta forma Bitey se concentra en el razonamiento y la generación de respuestas, mientras los servicios gestionan el acceso a los datos.

API pública

Toda integración utilizará la misma API.

`/api/v1/chat`

`/api/v1/customers`

`/api/v1/devices`

`/api/v1/services`

`/api/v1/tickets`

`/api/v1/knowledge`

`/api/v1/admin`

No habrá APIs específicas para WordPress o Flutter.

Todos los canales utilizarán exactamente la misma lógica.

Panel administrativo

El panel no será solo un CRUD.

Dispondrá de módulos como:

- Dashboard.
- Clientes.
- Dispositivos.
- Tickets.
- Servicios.
- Base de conocimiento.

- Bitey AI.
- Analítica.
- Configuración.
- Usuarios y permisos.

Plan de construcción definitivo

A partir de todo el análisis realizado, propongo organizar el desarrollo en **12 entregables** claramente definidos:

Fase	Objetivo
1	Foundation (core, configuración, seguridad, logging)
2	Database (migraciones, índices y diccionario de datos)
3	Identity (usuarios, roles y permisos)
4	Companies (multiempresa)
5	CRM (clientes, dispositivos y direcciones)
6	Communication (conversaciones y mensajes)
7	HelpDesk (tickets y seguimiento)
8	Service Catalog (categorías, servicios y precios)
9	Knowledge Base (artículos, sinónimos y traducciones)
10	Bitey AI Engine (motores, memoria y recomendaciones)
11	Integraciones (WordPress, Flutter y WhatsApp Cloud API)
12	Dashboard & Analytics (KPIs, informes y administración)

Mi recomendación final

En lugar de seguir generando código aislado, el siguiente documento que construiría sería la **Technical Design Specification (TDS)** de BiteFixes. La TDS convertirá